

Progetto di programmazione avanzata e parallela

Specifica di implementazione - Linguaggio C / OpenMP

Riduzione di Yang-Zhang per 23 Wang tiles

Generazione della regione, solver e scheduling parallelo

Scopo - Definire una specifica implementativa sufficientemente precisa da guidare lo sviluppo e i test, mantenendo separati: (i) il tileset fisso del paper, (ii) la costruzione della regione a partire da una istanza Cubic Monotone 1-in-3 SAT, (iii) il solver di tiling, (iv) la parallelizzazione e (v) il rendering diagnostico.

Elemento	Decisione di progetto
Linguaggio core	C17; OpenMP per il solver parallelo.
Tileset	I 23 Wang tiles unitari di Yang-Zhang; traslazioni soltanto, nessuna rotazione/riflessione.
Input principale	Istanza Cubic Monotone 1-in-3 SAT validata formalmente.
Output	Regione colorata, SAT/UNSAT del tiling, eventuale tiling testimone, statistiche e JSON diagnostico.
Renderer	Fork di xtraid/PAP_render, mantenuto disaccoppiato dal solver.
Principio di correttezza	Il solver non deve sostituire i 23 Wang tiles con i 14 generalized Wang tiles: questi ultimi sono solo macro-descrizioni utili per builder/debug.

Versione della specifica: 1.0 - 7 agosto 2026

1. Obiettivo, perimetro e criterio di successo

Il paper di Chao Yang e Zhujun Zhang dimostra che esiste un insieme fisso W di 23 Wang tiles per cui il problema di piastrellare regioni finite semplicemente connesse e con bordo colorato è NP-completo. La riduzione è da Cubic Monotone 1-in-3 SAT: per ogni formula ϕ viene costruita una regione D tale che ϕ è soddisfacibile se e solo se D è piastrellabile con W . Questa specifica traduce tale costruzione in una architettura software verificabile e parallelizzabile.

1.1 Deliverable primario

- Un eseguibile C che legga una formula valida, costruisca la regione D e la relativa colorazione del bordo secondo la riduzione del paper.
- Un tileset statico di 23 Wang tiles unitari, trascritto fedelmente dalla Figura 1 del paper e usato come unica sorgente di verità per il matching locale.
- Un solver reference, corretto prima che veloce, che usi soltanto regione + bordo + tileset e produca SAT/UNSAT ed eventualmente un tiling testimone.
- Un solver OpenMP che parallelizzi la ricerca senza permettere scritture concorrenti sulla stessa cella e che possa sfruttare un piano di zone/dependenze costruito dopo la generazione della regione.
- Un verificatore indipendente del testimone e un formato JSON comune per renderer e strumenti di test.

1.2 Non-obiettivi del primo milestone

- La trasformazione square $\rightarrow$ hex non fa parte del kernel C di questa specifica. Resta un ramo separato della roadmap e può riusare l'IR/JSON quando il core square è stabile.
- La variante a 29 Wang tiles e la riduzione a 111 rettangoli del paper non sono richieste.
- Z3 non sostituisce il solver C . Rimane utile come oracle/counterexample tool per istanze piccole, coerentemente con la roadmap.
- Il renderer non prende decisioni logiche e non viene linkato al solver: consuma solo output serializzato.

Gate 1 - Prima di iniziare l'ottimizzazione OpenMP deve essere possibile eseguire: formula $\rightarrow$ regione $\rightarrow$ solver reference $\rightarrow$ verifier, su almeno una istanza SAT e una UNSAT.

2. Modello formale implementato

2.1 Istanza sorgente: Cubic Monotone 1-in-3 SAT

L'input deve rispettare contemporaneamente tre condizioni: ogni clausola contiene esattamente tre occorrenze; tutte le occorrenze sono positive; ogni variabile compare esattamente tre volte nell'intera formula. La semantica di soddisfacimento richiede esattamente una occorrenza vera in ogni clausola. Il paper osserva che, con tali vincoli, il numero di variabili coincide con il numero di clausole.

Attenzione ai duplicati - Una clausola può contenere più volte la stessa variabile; l'esempio del paper usa x_1 due volte nella prima clausola. Il parser non deve imporre variabili distinte all'interno di una clausola.

2.2 Istanza di tiling

Una cella attiva della regione è un quadrato unitario. Ogni Wang tile ha quattro colori orientati N/E/S/W. Due celle ortogonalmente adiacenti sono compatibili se i colori dei lati in contatto coincidono. Se un lato di una cella appartiene al bordo della regione, il colore della tile su quel lato deve coincidere con il colore assegnato al corrispondente segmento di bordo. Le tile possono essere solo traslate.

2.3 I 23 Wang tiles e i 14 macro-tipi generalizzati

Il paper presenta i 23 Wang tiles unitari come gruppi obbligati da colori interni unici; questi gruppi possono essere interpretati come 14 generalized Wang tiles. A livello software è utile mantenere entrambe le viste, ma con ruoli diversi.

Famiglia macro	Varianti	Funzione nella riduzione
Variabili	V0, V1	Scelgono il valore della variabile; V0 è un bar verticale di altezza 3, V1 è una singola Wang tile.
Clausole	C0, C1	Controllano la condizione exactly-one nel blocco di clausola.
Forwarders	F0, F1	Propagano i segnali 0/1 da sinistra a destra.
Left anchors	L0, L1	Dal bordo inferiore guidano la posizione dello scambio.
Right anchors	R0, R1	Dal bordo superiore guidano la posizione dello scambio.
Crossovers	X00, X01, X10, X11	Scambiano due segnali su righe adiacenti, una variante per ogni coppia 00/01/10/11.

Regola di fedeltà - Il solver deve vedere 23 tile unitari. MacroKind e gadget metadata possono servire per costruzione, logging e test, ma non possono diventare tile indivisibili nel solver principale, altrimenti si implementerebbe un problema diverso.

3. Architettura generale



Figura 1 - Pipeline completa proposta.

La separazione in fasi evita di mescolare la logica della riduzione con la ricerca combinatoria. Il tileset è fisso; la regione dipende dalla formula; il piano di scheduling dipende dalla geometria generata; lo stato di ricerca è effimero e appartiene al solver.



Figura 2 - Dati fissi, dati per istanza e dati dinamici di ricerca.

Fase	Input	Output	Mutabilità
A. Static setup	codice sorgente	TILESET[23], tabelle colori, metadata	immutabile
B. Parse/validate	file formula	Formula	immutabile dopo validazione
C. Reduction	Formula	Region + SignalPlan + swap list	immutabile
D. Plan build	Region + metadata	TaskPlan / zone ownership	immutabile
E. Solve	Region + TaskPlan	SolveState / result	dinamico, per branch
F. Verify/export	result	verified solution + JSON	sola lettura del risultato

4. Compile time, preprocessing e runtime

4.1 Cosa puo essere statico nel binario

- L'enumerazione delle direzioni e la funzione opposite(dir).
- L'insieme dei colori e i 23 record WangTile, una volta trascritti esattamente dal paper.
- La mappatura opzionale tile_id -> macro family e posizione interna, solo diagnostica.
- I template a dimensione fissa per le parti della regione il cui layout e costante, se trascritti dal paper.

4.2 Cosa deve essere costruito per ogni formula

- Sequenza delle tre occorrenze di ogni variabile e righe ridondanti.
- Sequenza di destinazione delle occorrenze nelle clausole.
- Permutazione e lista di trasposizioni adiacenti.
- Dimensioni, celle attive e colori del bordo della regione D.
- Descrittori delle sub-regioni e ownership dei task.

4.3 Cosa esiste solo durante la ricerca

- Domini di tile possibili per ogni cella, assegnamenti parziali, trail di rollback, stack DFS.
- Copie di stato create ai punti di split parallelo.
- Flag globale di soluzione trovata, statistiche e buffer della soluzione vincente.

Conseguenza - La regione specifica della formula non puo essere costruita a compile time. Il punto corretto per calcolare zone e dipendenze e fra reduction builder e solve, cioe nel preprocessing della singola istanza.

5. Strutture dati C consigliate

5.1 Tileset

```
typedef uint8_t ColorId;
typedef uint8_t TileId;

typedef enum { DIR_N = 0, DIR_E, DIR_S, DIR_W } Dir;

typedef struct {
    TileId id;
    ColorId edge[4]; /* N, E, S, W */
    uint8_t macro_kind; /* debug only */
    int8_t macro_dx; /* debug only */
    int8_t macro_dy; /* debug only */
} WangTile;

extern const WangTile TILESET[23];
```

Non creare 23 tipi struct differenti. Una sola struct descrive una tile e un array statico contiene le 23 istanze. In questo modo matching, serializzazione e test sono uniformi.

5.2 Regione

```
typedef struct {
    int32_t x, y;
    int32_t neighbor[4]; /* -1 se lato di bordo */
    ColorId boundary[4]; /* COLOR_NONE se non e bordo */
    uint8_t active;
    uint32_t zone_id;
} Cell;

typedef struct {
    int32_t width, height;
```

```

    Cell *cells;          /* row-major, width*height */
    size_t active_count;
} Region;

```

Per questa riduzione una rappresentazione densa nel bounding box è semplice e veloce: il neighbor lookup è $O(1)$, la memoria è contigua e la verifica è cache-friendly. Il campo active permette comunque regioni poliminiche non rettangolari senza introdurre un grafo generale.

5.3 Dominio delle tile in 32 bit

Poiché $23 < 32$, l'insieme delle tile ammissibili per una cella può essere codificato in un `uint32_t`. Il bit `t` vale 1 se la tile `t` è ancora possibile. Questa scelta riduce molte operazioni di propagazione a AND, OR, test di zero e popcount.

```

#define TILE_COUNT 23u
#define DOMAIN_ALL ((1u << TILE_COUNT) - 1u)

typedef struct {
    uint32_t *domain;          /* un mask per cella */
    uint32_t *assignment;      /* opzionale / derivabile dal domain */
    /* trail per rollback seriale */
} SolveState;

```

5.4 Tabelle di compatibilità

```

uint32_t compat[4][23];
uint32_t boundary_mask[4][COLOR_COUNT];

```

`compat[d][t]` contiene il bitmask delle tile ammissibili nella cella vicina in direzione `d` quando la cella corrente usa `t`. Le tabelle vanno derivate automaticamente da `TILESET` durante l'inizializzazione: duplicare a mano sia `tileset` sia `compatibilità` creerebbe due sorgenti di verità.

6. Formato di input e validazione

È consigliato un formato testuale volutamente minimale, separato da spazi, per rendere semplice sia il parser C sia la generazione automatica di casi di test.

```

vars 3
clause 1 1 3
clause 2 2 3
clause 1 2 3

```

Le variabili sono numerate 1..n. I duplicati nella stessa clausola sono leciti. Non esiste sintassi per la negazione: un segno meno o un identificatore fuori range è errore di input.

Controllo	Errore se...
Header	vars manca, $n \leq 0$, o compaiono più/fewer clausole di n .
Arity	una clausola non contiene esattamente 3 identificatori.
Monotonia	un token rappresenta una negazione o non è un intero positivo.
Cubicità	una variabile compare un numero di volte diverso da 3.
Range	un identificatore non appartiene a 1..n.
EOF	restano token inattesi dopo l'ultima clausola.

Invariante utile - Dopo la validazione il numero totale di occorrenze è $3n$ e il numero di clausole è n . Da questo punto i moduli successivi possono assumere tali proprietà senza ricontrollarle ad ogni funzione.

7. Costruzione della riduzione

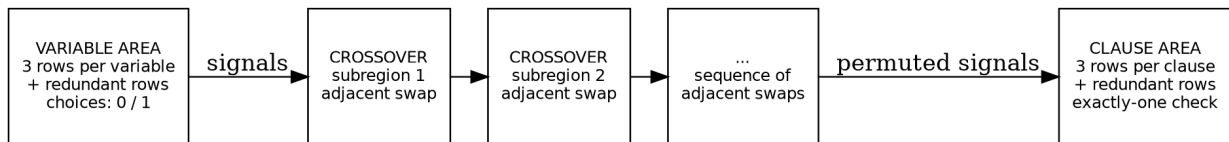


Figura 3 - Struttura logica della regione: variabili a sinistra, catena di crossover al centro, clausole a destra.

7.1 Token di segnale: distinguere le occorrenze

Per calcolare la permutazione non basta rappresentare un segnale come "x_i", perché le tre copie sono logicamente uguali ma geometricamente sono tre oggetti distinti. Ogni occorrenza deve quindi avere un ID univoco.

```
typedef enum { SIGNAL_VAR, SIGNAL_REDUNDANT } SignalKind;

typedef struct {
    SignalKind kind;
    uint32_t var_id;          /* valido per SIGNAL_VAR */
    uint8_t occurrence;       /* 0,1,2 */
    uint32_t token_id;        /* sempre univoco */
} SignalToken;
```

7.2 Sequenza sorgente

Con n variabili, la parte sinistra emette 3 segnali per variabile e inserisce una riga ridondante tra due gruppi consecutivi. L'altezza logica è quindi $h = 3n + (n-1) = 4n-1$. Le righe ridondanti possono essere fissate a 0 mediante i colori di bordo, come nel paper.

```
x1^0 x1^1 x1^2 z1 x2^0 x2^1 x2^2 z2 ... xn^0 xn^1 xn^2
```

7.3 Sequenza destinazione

La parte destra è organizzata per clausole: per ogni clausola si elencano le sue tre occorrenze nell'ordine delle tre righe della sub-regione; tra clausole consecutive si inserisce una riga ridondante. Per ogni variabile si assegna in modo stabile la prima, seconda e terza occorrenza sorgente alle tre occorrenze incontrate nel flattening delle clausole. In questo modo anche formule con duplicati producono una vera permutazione di token univoci.

7.4 Lista di trasposizioni adiacenti

La permutazione viene realizzata come sequenza di swap di elementi adiacenti. Un algoritmo di insertion/bubble stabile e sufficiente e produce al massimo $h(h-1)/2$ swap.

```
current = source
for pos in 0 .. h-1:
    desired = target[pos]
    j = index_of(desired, current, start=pos)
    while j > pos:
        emit swap(j-1, j)
        exchange current[j-1], current[j]
        j--
assert current == target
```

7.5 Crossover sub-region

Nel paper una sub-regione di larghezza w forza lo scambio dei segnali sulle righe w e $w+1$ (numerazione 1-based). Per l'esempio $w=8$ il bordo superiore è b^7 e quello inferiore è l^7 ; le ancore R e L si incontrano e costringono una tile crossover X nella posizione determinata dal bordo. Nel codice conviene mantenere `swap_row` in 0-based e convertire esplicitamente con `paper_w = swap_row + 1`, per evitare errori off-by-one.

7.6 Template fissi e dati da non inventare

I colori e la scomposizione esatta dei 23 Wang tiles devono essere trascritti dalla Figura 1; analogamente le geometrie fisse delle sub-regioni variabile/clausola devono seguire le figure del paper. La prosa del paper è sufficiente a definire la funzione dei gadget, ma non va usata per indovinare lati o colori interni non esplicitati nel testo. Il builder dovrebbe quindi caricare template statici verificati una volta contro le figure.

Gate 2 - Dato source e target, il test del modulo permutation deve verificare che applicando in ordine tutti gli swap si ottenga esattamente target. Il test è indipendente dal solver e deve passare prima di costruire celle e bordi.

8. Plan builder: zone indipendenti e dipendenze

La board non viene rappresentata come un grafo. Il solo oggetto a forma di DAG e il piano di esecuzione, costruito dopo la regione. Ogni cella riceve un `zone_id`; ogni `ZoneTask` possiede un insieme disgiunto di celle e descrive quali zone devono aver fissato il proprio bordo d'ingresso prima che il task possa procedere.

```
typedef enum {
    ZONE_VARIABLE,
    ZONE_CROSSOVER,
    ZONE_CLAUSE,
    ZONE_FORWARD_ONLY
} ZoneKind;

typedef struct {
    uint32_t id;
    ZoneKind kind;
    uint32_t first_cell; /* oppure lista/range compatto */
    uint32_t cell_count;
    uint32_t pred[4];
    uint8_t pred_count;
    uint32_t paper_swap_row; /* solo crossover, se utile */
} ZoneTask;

typedef struct {
    ZoneTask *task;
    uint32_t task_count;
} TaskPlan;
```

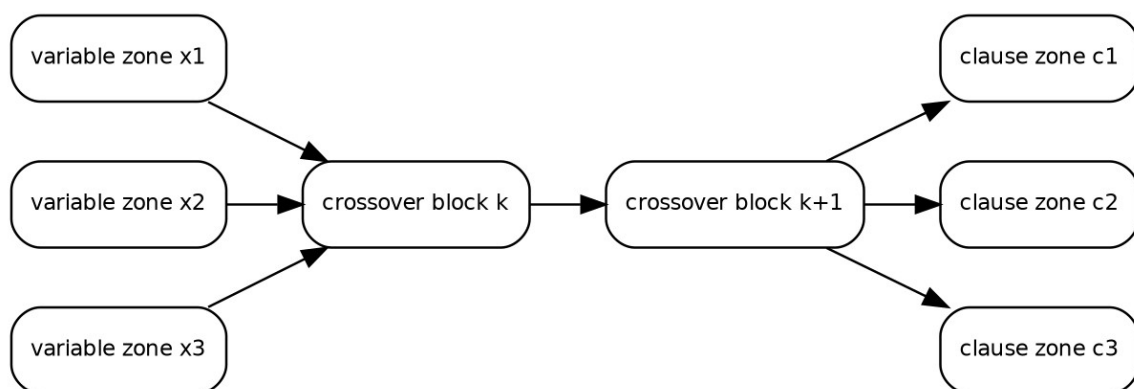


Figura 4 - Esempio concettuale di dipendenze. Le zone di clausola possono diventare eseguibili in parallelo quando i segnali finali sono fissati.

8.1 Ownership e assenza di race sulle celle

- Ogni cella attiva appartiene a una e una sola zona.
- Un task può scrivere solo celle della propria zona e solo nello stato del branch che sta elaborando.
- I dati di una zona predecessore diventano read-only quando vengono pubblicati alla successiva.

- Due task che potrebbero produrre alternative diverse non condividono lo stesso SolveState: lavorano su copie o su strutture copy-on-split.

8.2 Non confondere OpenMP schedule(dynamic) con il task scheduler

schedule(dynamic) distribuisce iterazioni di un omp for. Per una dipendenza irregolare fra zone e più naturale usare task OpenMP. Il runtime OpenMP decide dinamicamente quale worker esegue un task pronto; le clausole depend, oppure una coda esplicita di task pronti, esprimono il vincolo di precedenza. schedule(dynamic) resta utile per batch completamente indipendenti, per esempio una lista di branch già materializzati.

Granularità - Non creare un task per ogni cella. Il costo di scheduling diventerebbe comparabile o superiore al lavoro utile. Una zona, un crossover block o un sottoalbero di ricerca sono granularità appropriate; come euristica iniziale creare circa 8-32 task pronti per thread, non milioni di task minuscoli.

9. Solver reference: correttezza prima della parallelizzazione

Il solver reference deve ignorare la semantica SAT della formula. Riceve soltanto Region e TILESET. Questo è importante per dimostrare sperimentalmente che la regione generata è davvero una istanza di tiling e non semplicemente una codifica da cui il programma rilegge direttamente l'assegnamento.

9.1 Inizializzazione dei domini

1. Per ogni cella attiva porre domain = DOMAIN_ALL.
2. Per ogni lato di bordo applicare domain &= boundary_mask[dir][color].
3. Inserire in coda le celle il cui dominio è stato ristretto.
4. Eseguire propagazione locale fino a punto fisso oppure conflitto.

9.2 Propagazione locale

Se il dominio della cella i cambia, per ogni vicino j si eliminano dal dominio di j le tile che non hanno supporto in nessuna tile ancora possibile in i. Con 23 tile il controllo può iterare i bit settati e OR-are compat[dir][tile].

```
supported = 0;
for each tile t set in domain[i]:
    supported |= compat[dir][t];
new_domain_j = domain[j] & supported;
if new_domain_j == 0: conflict;
if new_domain_j != domain[j]: update + enqueue(j);
```

9.3 Scelta della decisione

Usare MRV: scegliere la cella con il minor numero di tile ancora possibili fra quelle con popcount > 1. A parità è utile una regola deterministica, per esempio maggior numero di vicini già ristretti e poi indice row-major. Questo rende il solver seriale riproducibile.

9.4 Rollback con trail

Nel DFS seriale evitare di copiare l'intera board ad ogni scelta. Prima di modificare domain[c], salvare (cell_idx, old_mask) in un trail. Ogni chiamata ricorsiva registra il mark iniziale del trail e, in backtrack, ripristina gli elementi fino a quel mark.

Gate 3 - Il solver reference deve validare sempre il proprio testimone con un secondo passaggio verify_solution() che controlla ogni cella, ogni adiacenza e ogni bordo. Nessun risultato SAT viene accettato solo perché il DFS è terminato senza conflitto.

10. Solver OpenMP: parallelizzazione della ricerca

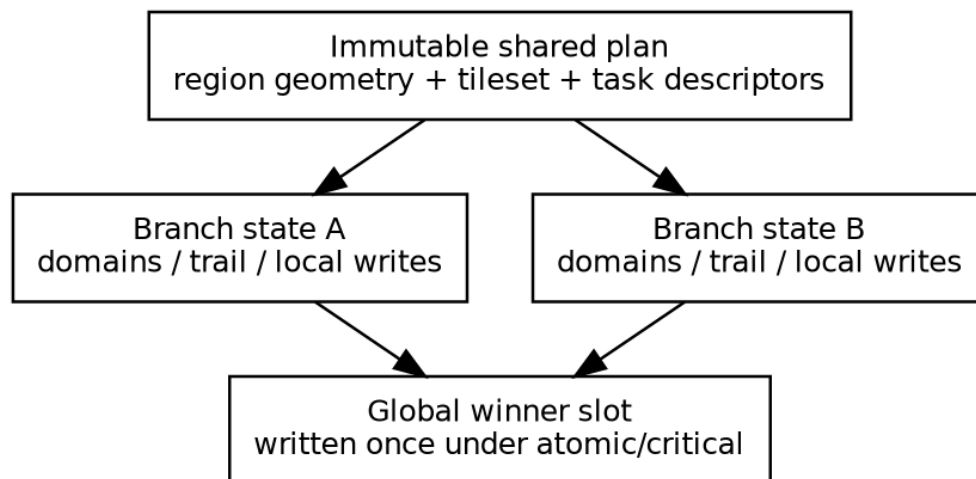


Figura 5 - Il piano è condiviso e immutabile; gli stati di branch non condividono scritture.

10.1 Strategia raccomandata: ibrida

La parallelizzazione più robusta combina due livelli. Il primo è il parallelismo dell'albero di ricerca: ai primi livelli del DFS si clonano pochi stati e si creano task indipendenti. Il secondo usa TaskPlan per scegliere punti di split e per eseguire zone disgiunte quando i vincoli di ingresso sono già fissati. In questo modo il programma ottiene parallelismo anche quando la catena centrale di crossover impone una forte dipendenza sinistra->destra.

10.2 Cutoff di creazione task

Creare task ricorsivamente senza limite produce overhead e consumo di memoria. Il solver deve avere parametri `split_depth` e `min_work`. Una politica iniziale semplice è generare task solo fino a quando il numero di branch pronti è circa 8-32 volte `omp_get_max_threads()`; sotto tale soglia ogni task continua in DFS seriale con trail locale.

10.3 Scheletro OpenMP

```
atomic_bool found = false;

#pragma omp parallel
{
    #pragma omp single
    {
        spawn_search_tasks(root_state, 0, plan, &found);
    }
}

/* Ogni task: */
if (!atomic_load(&found)) {
    SolveState local = clone_or_take(branch_state);
    if (dfs_serial(&local)) {
        #pragma omp critical(solution_commit)
        {
            if (!atomic_load(&found)) {
                save_solution(&local);
                atomic_store(&found, true);
            }
        }
    }
    destroy_state(&local);
}
```

10.4 Dipendenze fra zone

Le depend clauses possono essere usate quando i task sono coarse-grained e il numero di predecessori è piccolo. Un array di token per task è sufficiente a rappresentare la disponibilità del bordo d'ingresso. Se la gestione di depend dinamiche rende il codice meno portabile, è preferibile una coda di task pronti costruita dal plan builder: la correttezza non deve dipendere da una feature OpenMP particolarmente nuova.

10.5 Race condition reali da evitare

Risorsa	Rischio	Regola
domain/assignment	due branch scrivono la stessa cella	mai condividere SolveState mutabile fra branch.
global solution	piu thread trovano SAT quasi insieme	commit sotto critical/CAS; il primo vince.
found flag	lettura/scrittura non sincronizzata	C11 atomic o OpenMP atomic.
statistiche	incrementi persi	reduction per-thread o atomiche solo se necessario.
allocator/trail	contenzione o use-after-free	ownership per task; arena/thread-local se utile.
renderer/output file	scritture concorrenti	serializzare solo dopo la scelta del winner.

10.6 Cosa non fare

- Non fare lavorare piu thread sulla stessa board globale con lock per cella: il lock traffic elimina gran parte del vantaggio e rende il backtracking molto difficile da ragionare.
- Non alternare continuamente regioni omp parallel / single per ogni crossover: si paga ripetutamente il costo di sincronizzazione. Tenere una sola parallel region esterna e creare task al suo interno.
- Non usare il significato logico di x_i per dichiarare direttamente una formula SAT/UNSAT nel solver tiling; ciò bypasserebbe l'istanza geometrica.

11. Complessità e costanti pratiche

La NP-completezza implica che non va promesso un tempo polinomiale del solver generale. E però utile separare il costo polinomiale della riduzione dal costo esponenziale della ricerca e quantificare le costanti controllabili.

Componente	Stima consigliata	Osservazione
Validazione formula	$O(n)$	$3n$ occorrenze totali.
Segnali	$h = 4n-1$	Tre righe per variabile + $n-1$ ridondanti.
Swap list	$O(h^2)$ tempo / swap nel caso peggiore	Insertion/bubble stabile.
Costruzione esplicita regione	polinomiale; upper bound conservativo $O(h^4)$ celle per una rasterizzazione ingenua	Ogni crossover ha altezza $O(h)$, larghezza $\leq h$ e possono essercene $O(h^2)$. La costruzione reale può essere molto più piccola.
Compatibilità tileset	$4 * 23 * 23$ confronti	Costante, trascurabile.
Dominio per cella	4 byte	23 bit usati in uint32_t.
DFS tiling	esponenziale nel worst case	MRV + propagation riducono il ramo, non la classe.
Task overhead	non asintotico ma rilevante	Da ammortizzare con task coarse-grained.

11.1 Stima delle costanti del matching

Una propagazione su un arco cella-vicino richiede al massimo 23 iterazioni sui bit del dominio sorgente e poche operazioni bitwise. Questo è abbastanza piccolo da rendere conveniente un solver bitset. Il collo di bottiglia sarà il numero di stati esplorati e il costo delle copie ai punti di split, non il confronto di colori in se.

11.2 Parallel speedup atteso

La catena di crossover encode una permutazione come sequenza di trasposizioni, quindi una parte del flusso è intrinsecamente ordinata. Il parallelismo più abbondante tende a trovarsi fra branch di ricerca indipendenti e, dopo che gli ingressi sono fissati, fra alcune sub-regioni disgiunte. Per questo il progetto non deve basare la propria scalabilità soltanto sul parallelismo geometrico della board.

12. Integrazione con PAP_render

Il repository PAP_render è un renderer CLI Python già testato, con pipeline separata in Palette, VirtualVRAM, SceneParser, Blitter e RenderingPipeline. Il fork deve riusare il principio di pipeline e l'export PNG, ma non è opportuno forzare il nuovo dominio dentro i vincoli originali del renderer: tile_map 15x20, frame 640x480, 64 tile 32x32 e palette indicizzata a 16 colori.

12.1 Contratto C -> renderer

```
{
  "schema": "wang23-solution-v1",
  "geometry": "square",
  "width": 123,
  "height": 47,
  "cells": [
    {"x":0, "y":0, "active":true, "tile_id":7, "zone":3}
  ],
  "boundary": [
    {"x":0, "y":0, "dir":"w", "color":"v"}
  ],
  "meta": {
    "status":"SAT",
    "threads":8,
    "formula":"example.cm13"
  }
}
```

12.2 Modifiche raccomandate al fork

- Mantenere la lettura palette e l'export PNG come concetti riusabili.
- Sostituire il SceneParser vincolato a tile_map 15x20 con un parser dell'IR sopra.
- Aggiungere un WangAssetFactory che generi automaticamente una tile diagnostica da tile_id e quattro edge colors; non disegnare manualmente 23 asset.
- Rendere frame size dipendente dal bounding box della regione e da un parametro pixel_per_cell.
- Disabilitare sprites/UI nell'MVP Wang; possono restare nel branch originale ma non servono alla demo teorica.
- Non comprimere colori Wang logicamente distinti solo per rientrare nella palette originale a 16 indici. Se i colori logici eccedono 16, usare una palette diagnostica dinamica o un path RGB separato.

12.3 Preparazione al ramo hex

Il formato intermedio deve includere geometry. In futuro geometry="hex-axial" potrà sostituire x,y con q,r e il fork potrà implementare axial->pixel, come previsto dalla roadmap, senza modificare il solver square o importare Python nel core C.

Gate 4 - Il renderer deve poter ricevere un JSON di 10-20 celle e produrre un PNG corretto senza linkare il solver e senza leggere strutture C private.

13. Interfaccia da linea di comando

Separare i comandi permette di ispezionare ogni fase della riduzione e riprodurre i bug senza dover rilanciare l'intera pipeline.

```
wang23 validate formula.cm13
wang23 reduce formula.cm13 --region region.json --plan plan.bin
wang23 solve region.json --solver reference --solution sol.json
wang23 solve region.json --plan plan.bin --solver omp --threads 8 --solution sol.json
wang23 verify region.json sol.json
wang23 run formula.cm13 --solver omp --threads 8 --emit-render render.json
```

Opzione	Significato
--threads N	Numero massimo di thread OpenMP; default <code>omp_get_max_threads()</code> .
--solver reference omp	Seleziona baseline seriale o solver parallelo.
--split-depth D	Cutoff esplicito per task di ricerca; opzionale.
--stats file.json	Esporta nodi DFS, propagazioni, task creati, copie di stato, tempo per fase.
--no-plan	Per test: il solver OMP ignora metadata strutturali e parallelizza solo il search tree.
--deterministic	Forza 1 thread e tie-break stabili per debugging.

13.1 Exit status

Per evitare di trattare UNSAT come errore di processo, i comandi solve/run possono terminare con exit code 0 sia per SAT sia per UNSAT e stampare lo stato in output strutturato. Codici non-zero sono riservati a input invalido, I/O o errori interni.

14. Error handling e gestione risorse

Seguendo il livello di rigore del progetto di esempio, ogni errore prevedibile deve essere gestito esplicitamente e non deve trasformarsi in segmentation fault o assert di produzione.

Categoria	Comportamento richiesto
Parse/validation	Messaggio con file, riga/token e motivo; nessuna regione parziale lasciata viva.
Allocazione	Controllare malloc/calloc/realloc; cleanup centralizzato e ritorno di errore.
Region builder	Verificare overflow di <code>size_t</code> prima di <code>width*height</code> e prima di allocazioni.
Solver	Un conflitto logico e normale backtrack, non un errore.
OpenMP	Ogni stato ha owner chiaro; nessun puntatore a trail locale sopravvive al task.
Output	Scrittura atomica consigliata: file temporaneo + rename per solution/render JSON.

```
typedef enum {
    W23_OK = 0,
    W23_ERR_PARSE,
```

```

W23_ERR_INVALID_INSTANCE,
W23_ERR_IO,
W23_ERR_OOM,
W23_ERR_INTERNAL
} W23Status;

```

15. Struttura della repository C

```

wang23/
├── Makefile
├── include/
│   ├── tileset.h
│   ├── formula.h
│   ├── region.h
│   ├── reduction.h
│   ├── schedule.h
│   ├── solver.h
│   ├── verify.h
│   └── export.h
├── src/
│   ├── main.c
│   ├── tileset.c
│   ├── formula.c
│   ├── permutation.c
│   ├── region.c
│   ├── reduction.c
│   ├── schedule.c
│   ├── solver_ref.c
│   ├── solver_omp.c
│   ├── verify.c
│   └── export_json.c
├── tests/
│   ├── test_formula.c
│   ├── test_tileset.c
│   ├── test_permutation.c
│   ├── test_region.c
│   ├── test_solver.c
│   └── fixtures/
├── tools/
│   └── z3_oracle.py          # opzionale, solo cross-check
├── docs/
│   └── reduction_notes.md

```

15.1 Regole di dipendenza fra moduli

- tileset non dipende da formula, reduction, solver o renderer.
- formula non dipende da geometry o solver.
- reduction dipende da formula + region + tileset metadata, ma non dal solver.
- solver dipende da region + tileset + schedule, ma non dalla Formula semantica.
- verify dipende solo da region + tileset + soluzione.
- export legge oggetti pubblici e produce JSON; nessun modulo C dipende da PAP_render.

16. Piano di test e criteri di accettazione

16.1 Test del tileset

- Esattamente 23 tile ID, senza duplicati di ID.
- Ogni ColorId valido; opposite matching verificato per tutte le coppie compat dichiarate.
- Test di regressione con una rappresentazione testuale/hash del tileset trascritto dalla Figura 1, per rilevare modifiche accidentali.
- Macro metadata: 14 macro-tipi complessivi; solo test/diagnostica, non solver.

16.2 Test della riduzione

- Per n variabili, $h == 4*n - 1$.

- source e target contengono esattamente gli stessi token_id.
- Applicando la swap list a source si ottiene target.
- Le righe ridondanti sono fissate a 0 nei punti previsti.
- La regione generata e connessa. Aggiungere anche un controllo di assenza di buchi tramite flood-fill dell'esterno del bounding box per diagnosticare errori del builder.

16.3 Test solver

- SAT minimo noto: la formula del paper $x_1x_1x_3 / x_2x_2x_3 / x_1x_2x_3$; l'assegnamento $x_1=0, x_2=0, x_3=1$ soddisfa exactly-one.
- UNSAT piccolo: due clausole (x_1,x_1,x_2) e (x_1,x_2,x_2) ; ogni variabile compare esattamente tre volte ma nessun assegnamento soddisfa entrambe.
- reference e omp devono concordare su SAT/UNSAT per tutte le fixture piccole.
- Ogni soluzione restituita deve passare `verify_solution()`.
- Con piu tiling possibili, non confrontare byte-per-byte le board parallele: confrontare validita e stato SAT/UNSAT.

16.4 Test di concorrenza e performance

- Eseguire ripetutamente con `OMP_NUM_THREADS = 1,2,4,8` e verificare assenza di crash o risultati invalidi.
- Compilazione debug con AddressSanitizer + UndefinedBehaviorSanitizer; ThreadSanitizer solo se la toolchain OpenMP scelta lo supporta in modo affidabile.
- Misurare separatamente parse, reduction, plan, solve, verify, export. Il tempo di rendering non entra nel benchmark del solver.
- Registrare `task_created`, `state_clones`, `dfs_nodes`, `propagation_updates` e `max_task_depth` per capire se l'overhead e eccessivo.

Criterio di accettazione - Una ottimizzazione parallela e accettata solo se mantiene gli stessi risultati del reference e se il verifier approva il testimone. Lo speedup da solo non e una prova di correttezza.

17. Sequenza di implementazione consigliata

1. **Tileset + matching:** Trascrivere i 23 Wang tiles; implementare color matching, boundary masks e test.
2. **Parser formula:** Implementare formato .cm13 e validazione cubic/monotone/exact arity.
3. **Signal/permutation:** Generare token univoci, source/target e swap list; testare senza geometria.
4. **Region builder:** Implementare template di variabile/clausola e crossover parametrico; aggiungere boundary colors.
5. **Verifier:** Prima del solver: funzione che valida un tiling completo.
6. **Reference solver:** Bitmask domains + propagation + MRV + trail.
7. **Cross-check:** Confrontare piccole istanze con brute-force SAT o Z3 oracle.
8. **TaskPlan:** Annotare zone, ownership e dipendenze; nessuna parallelizzazione ancora.
9. **OpenMP tree split:** Parallelizzare branch indipendenti con una sola parallel region.
10. **Structural scheduling:** Usare TaskPlan per pruning/priority e zone realmente indipendenti.
11. **IR + renderer fork:** Esportare JSON e adattare PAP_render.
12. **Benchmark + relazione:** Misurare speedup, overhead e limiti; documentare senza claim non supportati.

18. Esempio end-to-end minimo

Formula usata nel paper:

```
vars 3
clause 1 1 3
```

```
clause 2 2 3
clause 1 2 3
```

Dopo la validazione: $n=3$, $h=11$. La sequenza sorgente logica e:

```
x1^0 x1^1 x1^2 z1 x2^0 x2^1 x2^2 z2 x3^0 x3^1 x3^2
```

La sequenza di destinazione segue le occorrenze nelle tre clausole, con le righe ridondanti nelle stesse separazioni fra blocchi. Il permutation builder produce una sequenza di swap adiacenti; il region builder concatena i corrispondenti crossover sub-regions fra parte variabile e parte clausola.

Il solver reference deve trovare SAT e il verifier deve accettare il testimone. La semantica SAT permette un controllo indipendente: $x_1=0$, $x_2=0$, $x_3=1$ rende ciascuna delle tre clausole exactly-one. Questo assegnamento e un oracle di test, non un input privilegiato del solver tiling.

19. Requisiti di consegna del modulo C

- Il progetto deve compilare con make su Linux recente; comando consigliato: `cc -std=c17 -O3 -Wall -Wextra -Wpedantic -fopenmp`.
- Nessun warning del compilatore nella configurazione di consegna.
- Ogni funzione non banale deve descrivere input, output, ownership e condizioni di errore; per funzioni concorrenti anche thread-safety/ownership.
- Tutte le risorse allocate devono essere rilasciate anche su error path.
- Input invalidi devono produrre errore leggibile e exit code non-zero, mai segmentation fault.
- Il solver parallelo deve funzionare anche con un solo thread e produrre una soluzione verificabile.
- Il tileset trascritto dal paper deve essere centralizzato in un solo file sorgente; nessuna copia divergente nei test o nel renderer.
- Il renderer e opzionale per la correttezza del solver ma richiesto per la demo visuale del progetto complessivo.

Appendice A - API minima suggerita

```
W23Status formula_load(const char *path, Formula *out, W23Error *err);
W23Status formula_validate(const Formula *f, W23Error *err);

W23Status reduction_build(const Formula *f, Region *r, ReductionMeta *m, W23Error *err);
W23Status taskplan_build(const Region *r, const ReductionMeta *m, TaskPlan *p, W23Error *err);

SolveResult solve_reference(const Region *r, const WangTile tiles[23], SolveOptions opt);
SolveResult solve_openmp(const Region *r, const TaskPlan *p, const WangTile tiles[23],
SolveOptions opt);

bool verify_solution(const Region *r, const WangTile tiles[23], const TileId *assignment,
W23Error *err);

W23Status export_region_json(const Region *r, const ReductionMeta *m, const char *path);
W23Status export_solution_json(const Region *r, const SolveResult *s, const char *path);
```

Appendice B - Invarianti da usare come assert di sviluppo

- `TILE_COUNT == 23` e ogni `assignment` $e < 23$.
- Per ogni cella attiva e direzione, `neighbor[d] >= 0 XOR boundary[d] != COLOR_NONE`.
- Se `neighbor[d] = j`, allora `cells[j].neighbor[opposite(d)]` punta alla cella corrente.
- Ogni cella attiva appartiene a esattamente una zona.
- Ogni token di source compare esattamente una volta in target.
- Dopo l'applicazione degli swap: `current == target`.
- Un `SolveResult SAT` deve avere un `tile_id` definito per ogni cella attiva e `verify_solution == true`.

Appendice C - Fonti e ruolo nel progetto

[1] C. Yang, Z. Zhang, “NP-completeness of Tiling Finite Simply Connected Regions with a Fixed Set of Wang Tiles”, arXiv:2405.01017v2, 2024. Fonte primaria per i 23 Wang tiles, i 14 generalized macro-tiles e la riduzione da Cubic Monotone 1-in-3 SAT.

- [2] C. Moore, J. M. Robson, “Hard Tiling Problems with Simple Tiles”, 2001. Fonte indicata nella bibliografia del progetto per la NP-completezza della variante planar/cubic monotone 1-in-3 SAT.
- [3] “Wang / Hex project - Roadmap MVP”, documento di pianificazione fornito. Usato per mantenere separati core teorico, solver/oracle, renderer e ramo square->hex; il procedural generator resta non necessario al nucleo.
- [4] “Progetto di programmazione avanzata e parallela - Parte 1 di 2 - Linguaggio C”, consegna di esempio fornita. Usato come riferimento per livello di dettaglio: sintassi, API, error handling, requisiti e casi d'uso espliciti.
- [5] xtraid/PAP_render, repository GitHub. Renderer CLI Python con Palette, VirtualVRAM, SceneParser, Blitter e RenderingPipeline; il fork è destinato al rendering diagnostico, non alla logica del solver.

Appendice D - Decisioni architetturali riassunte

Domanda	Decisione
Struct o grafo per la board?	Struct + array row-major; nessun grafo generale per le celle.
23 struct diverse?	No: un tipo WangTile e un array const di 23 record.
Generalized Wang tiles?	Solo macro metadata/template; il solver usa i 23 unitari.
Regione a compile time?	No: dipende dalla formula e viene generata per istanza.
Scheduling quando?	Dopo la regione, prima del solve.
OpenMP static/dynamic?	Task OpenMP per DAG/branch irregolari; schedule(dynamic) solo per loop indipendenti.
Come evitare race?	Ownership disgiunto delle celle e SolveState separato per branch; commit globale della sola soluzione vincente.
Granularità task?	Sub-regioni o sottoalberi, mai singole celle.
Renderer?	Processo separato via JSON; fork PAP_render.

Addendum architetturale

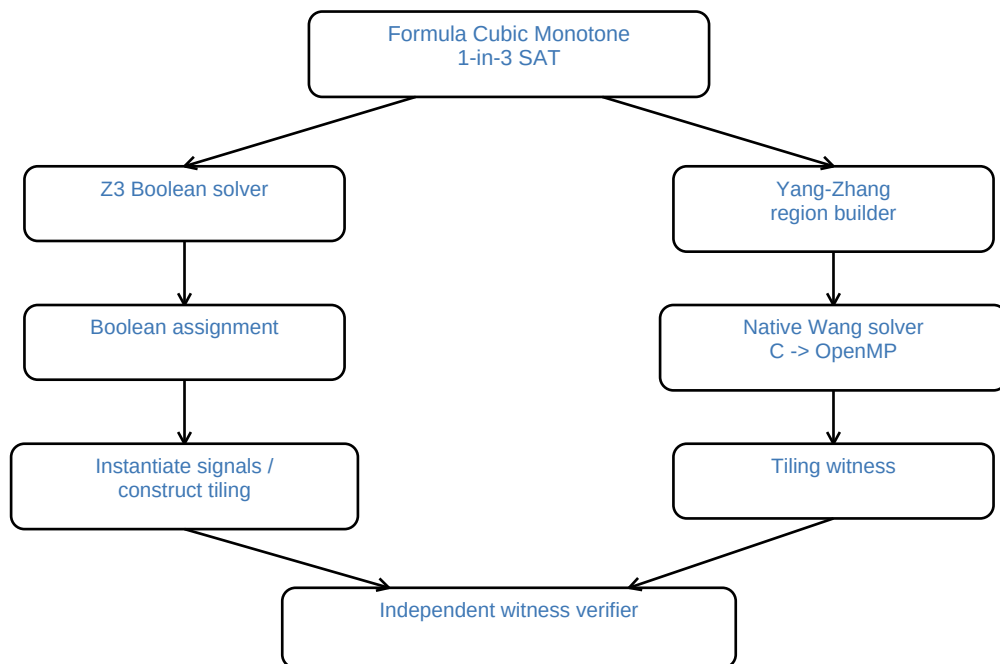
Doppio solver, validazione bidirezionale e traduzione square -> hex

Questo addendum integra la specifica già stampata: Z3 non è un componente opzionale, ma il solver di riferimento con cui il solver nativo delle Wang tiles viene confrontato. La riduzione viene verificata in entrambe le direzioni, separando sempre ricerca, verifica del testimone e rendering.

Principio di progetto - due solver indipendenti, un solo insieme di invarianti.

Nessun risultato è considerato valido perché prodotto da un particolare solver: ogni testimone deve passare un verifier indipendente e, per le istanze piccole, i due percorsi devono concordare almeno su SAT/UNSAT.

1. Architettura di cross-validation



A) Z3 witness -> tiling -> verifier

B) native tiling -> Boolean witness -> Z3/direct verifier

Terzo controllo consigliato (istanze piccole)

Aggiungere anche un encoding Z3 del tiling: stessa regione, una variabile tile-id per cella, matching orientato e boundary constraints. Z3-tiling e solver nativo non devono produrre lo stesso modello, ma devono concordare su SAT/UNSAT e ogni modello deve passare lo stesso verifier.

2. Square Wang -> hex Wang: effetto sull'architettura

Per l'MVP il solver C può rimanere un solver a quattro lati. Dopo aver trovato una pavimentazione square valida, una fase Python traduce ogni tile in una tile esagonale: i quattro colori significativi vengono riportati su quattro lati dell'esagono e i due lati aggiuntivi ricevono lo stesso colore helper. Questa fase produce la maschera/JSON per PAP_render.



Condizione necessaria - questa separazione è corretta solo se la trasformazione square->hex è dimostrata semantics-preserving. Il rendering da solo non basta: serve almeno un hex verifier che controlli i sei lati e garantisca che i due lati helper non introducano tilings spurie. Un vero solver hex a sei lati resta fuori dall'MVP.